МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «Информационных технологий»

Лабораторная работа № 3
по дисциплине «Введение в информационные технологии»
по теме: «**ФУНКЦИИ В PYTHON И БАЗОВЫЕ АЛГОРИТМЫ**»

Выполнил студент группы:
БВТ2601
Гутаев Эльдар Назирович
Проверил:

Москва 2026

Оглавление

Введение……3

Выполнение 4

Вывод..... 17

Введение

Цель и задачи работы:

Целью данной лабораторной работы является: изучить принципы создания и использования функций в языке Python, научиться передавать данные в функции и возвращать результаты, понять области видимости переменных; освоить работу со списками и кортежами, срезами и встроенными функциями; получить практический опыт реализации базовых алгоритмов обработки последовательностей и применения lambda-функций.

Задачи: В ходе выполнения лабораторной работы необходимо научиться разбивать программу на логически завершённые функции, использовать параметры и возвращаемые значения, обрабатывать последовательности данных, а также самостоятельно реализовывать простейшие алгоритмы поиска, подсчёта и агрегирования.

Выполнение

Задание 1. На рисунке 1: созданы функции greet(name), square(number) и max_of_two(a, b); каждая вызвана два раза с разными аргументами. Функции greet() выводят приветствия для имён «Иван» и «Мария», square(5) возвращает 25, square(-3) возвращает 9, max_of_two(7, 12) возвращает 12, а max_of_two(9.5, 3.2) возвращает 9.5. Функция max_of_two() реализована с помощью условного оператора if без встроенной функции max().

```
def greet(name):
    print(f"Здравствуйте, {name}!")


def square(number):
    return number ** 2


def max_of_two(a, b):
    if a > b:
        return a
    return b


greet("Иван")
greet("Мария")
print(square(5))
print(square(-3))
print(max_of_two(7, 12))
print(max_of_two(9.5, 3.2))
```

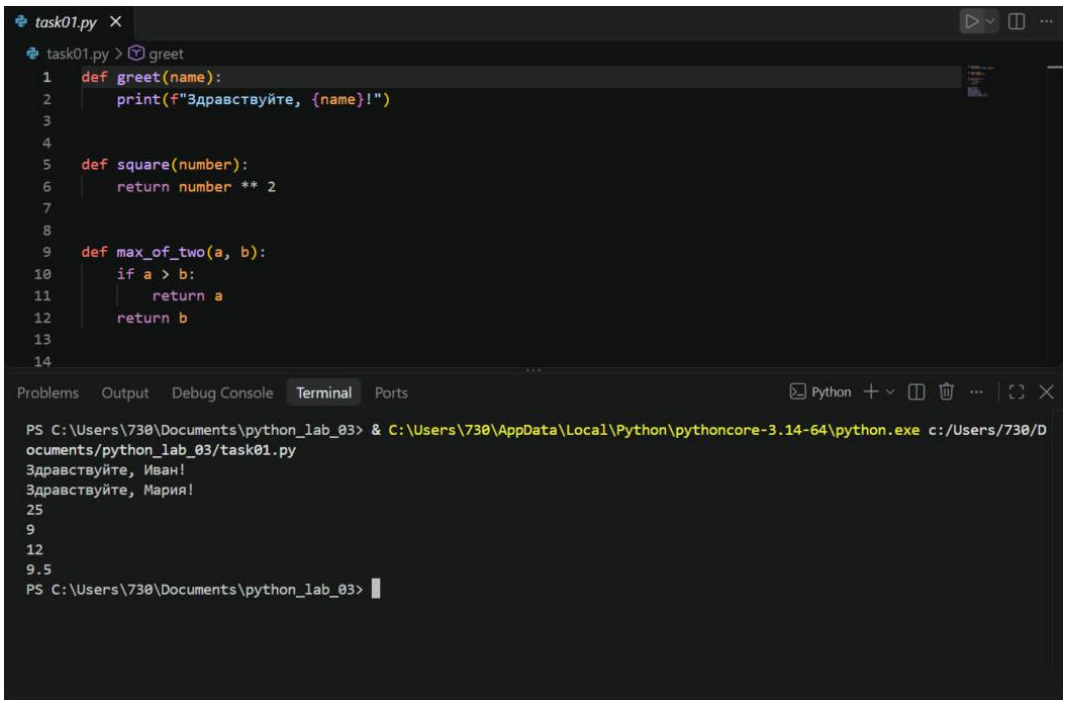


Рисунок 1. Создание и вызов функций

Задание 2. На рисунке 2: функция calculate_rectangle(5, 3) возвращает площадь прямоугольника (15), а calculate_perimeter(5, 3) возвращает периметр (16). Функция describe_rectangle() имеет параметр unit со значением по умолчанию «см»: при вызове без этого аргумента выводится единица «см», а при вызове describe_rectangle(5, 3, "м") используется переданное значение «м».

```
def calculate_rectangle(width, height):
    return width * height


def calculate_perimeter(width, height):
    return 2 * (width + height)


def describe_rectangle(width, height, unit="см"):
    print(f"Прямоугольник {width} x {height} {unit}")


print(calculate_rectangle(5, 3))
print(calculate_perimeter(5, 3))
describe_rectangle(5, 3)
describe_rectangle(5, 3, "м")
```

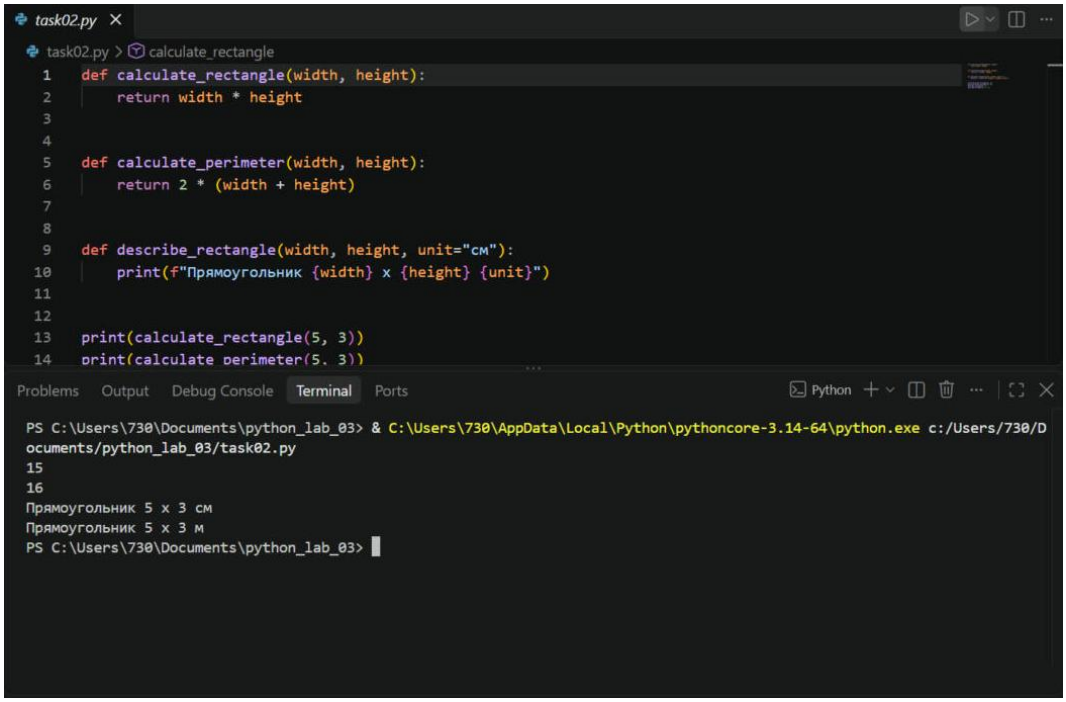


Рисунок 2. Аргументы и возвращаемые значения

Задание 3. На рисунке 3: программа выводит сначала 20, затем 10. Внутри функции example() создаётся локальная переменная value, которая существует только во время выполнения функции и скрывает глобальную

переменную с тем же именем. Присваивание value = 20 не изменяет глобальную переменную, поэтому вне функции выводится прежнее значение 10. На рисунке 3.1: в функцию добавлена ещё одна локальная переменная local_value = 30. Внутри функции она выводится (30), но обращение к ней после вызова функции вызывает ошибку NameError: name 'local_value' is not defined, так как локальная переменная перестаёт существовать после завершения функции и недоступна в глобальной области видимости.

```
value = 10


def example():
    value = 20
    print(value)


example()
print(value)
```

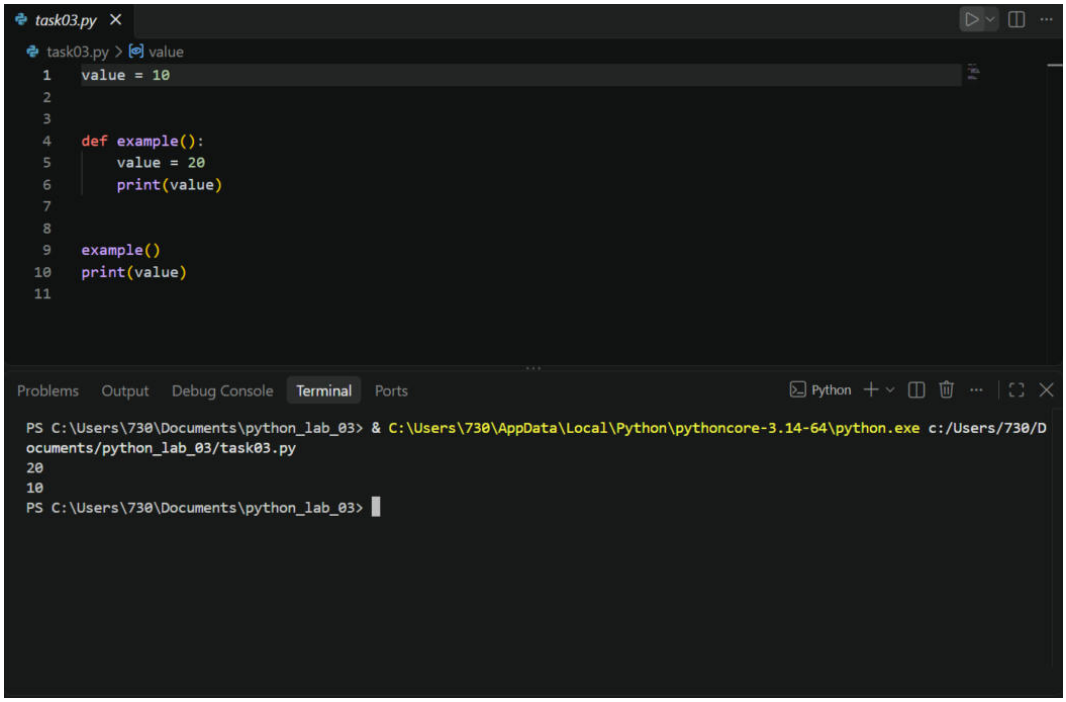


Рисунок 3. Локальная и глобальная переменные

```
value = 10


def example():
    value = 20
    local_value = 30
    print(value)
    print(local_value)


example()
print(value)
print(local_value)
```

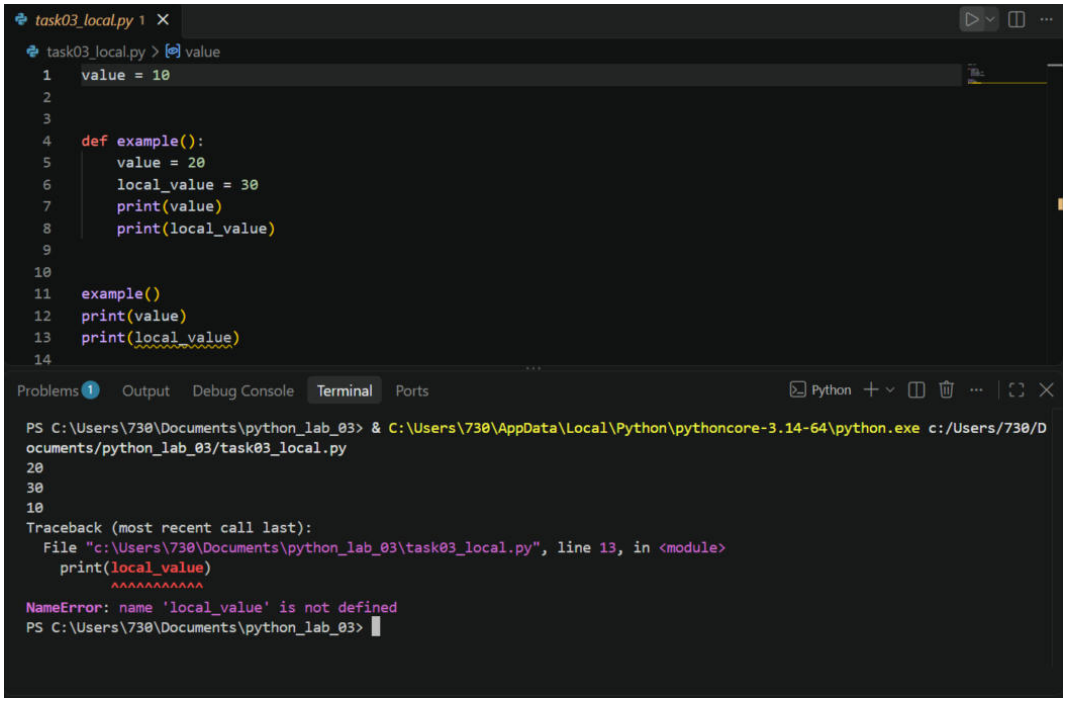


Рисунок 3.1. Обращение к локальной переменной вне функции

Задание 4. На рисунке 4: создан список из 10 целых чисел. Выведены первый (4), третий (15) и последний (12) элементы. Значение второго элемента изменено на 100, в конец списка добавлено число 55 методом append(), число 15 удалено методом remove(). Итоговый список содержит 10 элементов: [4, 100, 16, 23, 42, 7, 19, 31, 12, 55].

```
numbers = [4, 8, 15, 16, 23, 42, 7, 19, 31, 12]
print("Первый:", numbers[0])
print("Третий:", numbers[2])
print("Последний:", numbers[-1])
numbers[1] = 100
numbers.append(55)
numbers.remove(15)
print("Итоговый список:", numbers)
print("Количество элементов:", len(numbers))
```

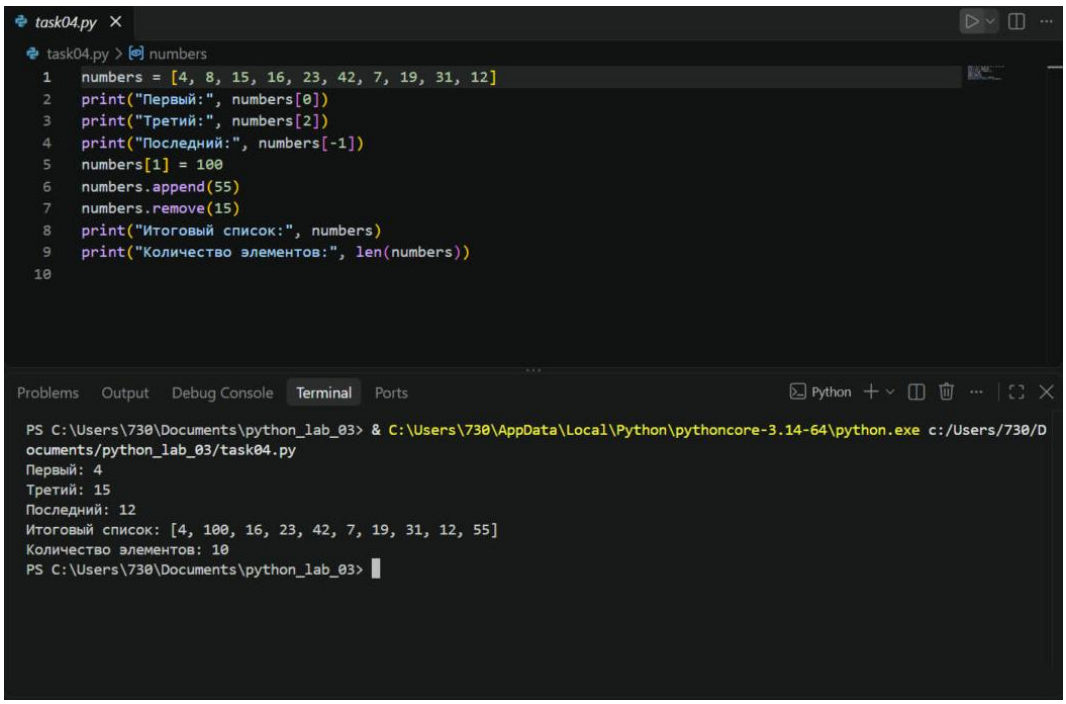


Рисунок 4. Работа со списками

Задание 5. На рисунке 5: с помощью срезов из списка чисел от 1 до 10 получены первые три элемента ([1, 2, 3]), последние три элемента ([8, 9, 10]), элементы с 3-го по 7-й ([3, 4, 5, 6, 7]), каждый второй элемент ([1, 3, 5, 7, 9]) и список в обратном порядке. Правая граница среза не входит в результат, поэтому для получения элементов с 3-го по 7-й использован срез numbers[2:7].

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print("Первые три:", numbers[:3])
print("Последние три:", numbers[-3:])
print("С 3-го по 7-й:", numbers[2:7])
print("Каждый второй:", numbers[::2])
print("В обратном порядке:", numbers[::-1])
```

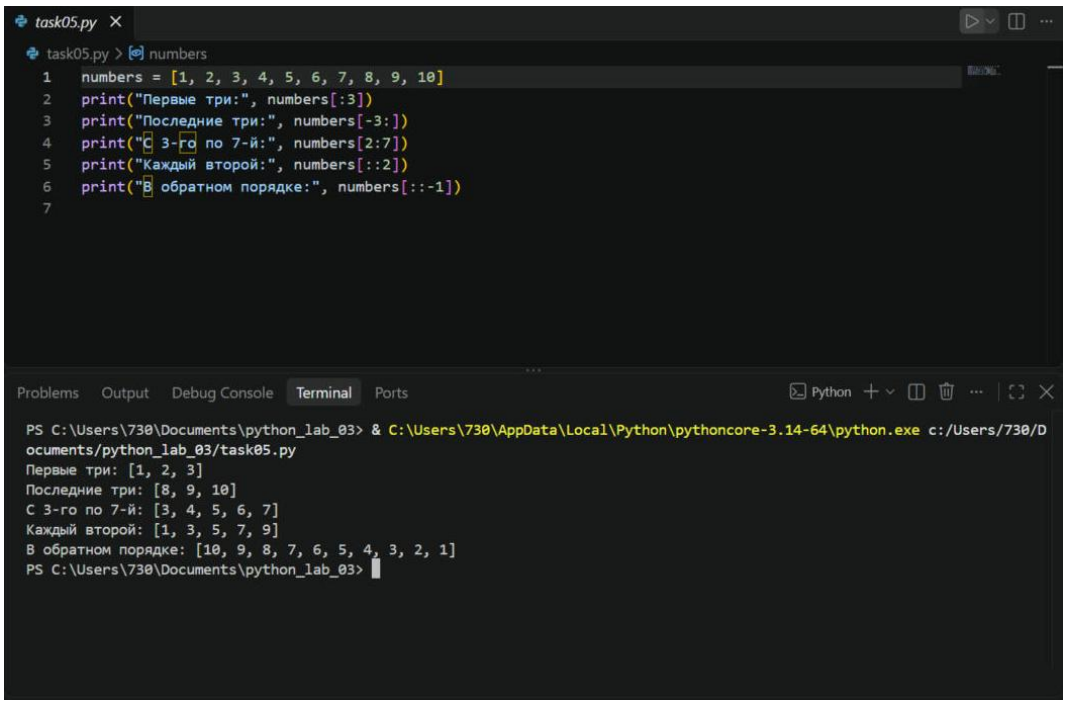


Рисунок 5. Срезы списков

Задание 6. На рисунке 6: создан кортеж measurements из 8 числовых значений. Выведены первый (12.5) и последний (15.0) элементы, а также срез [2:5] — (13.8, 15.2, 12.9). Первые два значения распакованы в переменные first и second. Для изменения значения кортеж преобразован в список, изменён первый элемент и результат преобразован обратно в кортеж. На рисунке 6.1: попытка изменить элемент кортежа напрямую вызывает ошибку TypeError: 'tuple' object does not support item assignment. Изменение невозможно, так как кортеж является неизменяемой последовательностью: после создания его элементы нельзя заменить, добавить или удалить.

```
measurements = (12.5, 14.1, 13.8, 15.2, 12.9, 14.7, 13.3, 15.0)
print("Первый:", measurements[0])
print("Последний:", measurements[-1])
print("Срез [2:5]:", measurements[2:5])

first, second = measurements[:2]
print("Распаковка:", first, second)

as_list = list(measurements)
as_list[0] = 10.0
measurements = tuple(as_list)
print("Новый кортеж:", measurements)
```

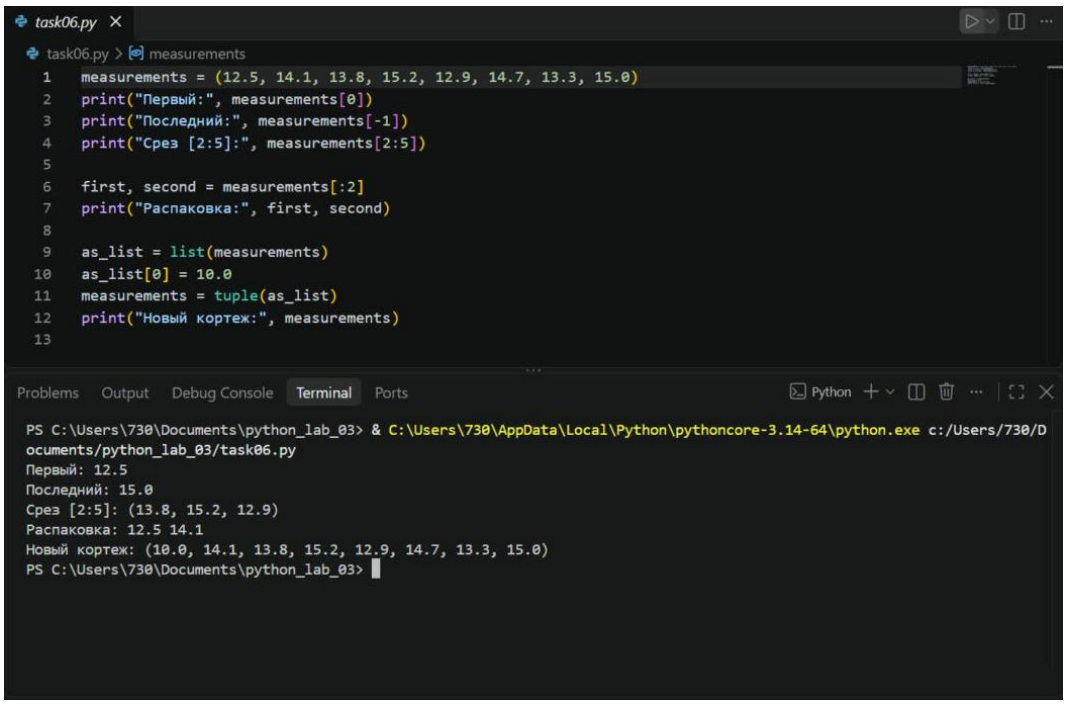


Рисунок 6. Работа с кортежем

measurements = (12.5, 14.1, 13.8, 15.2, 12.9, 14.7, 13.3, 15.0)
measurements[0] = 10.0

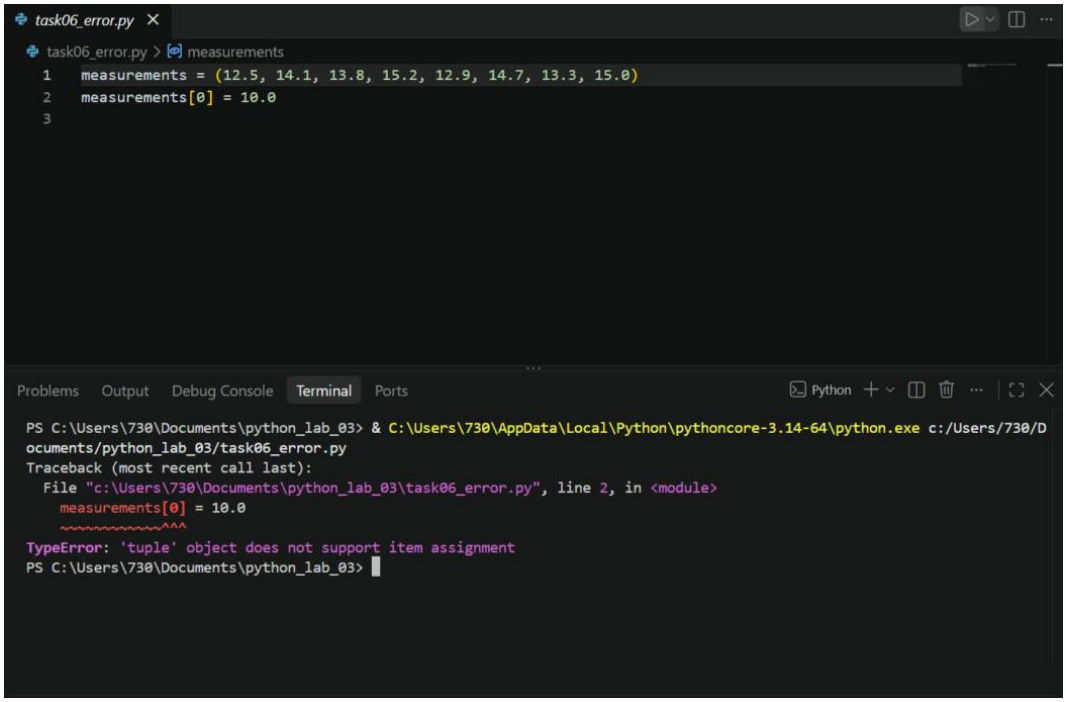


Рисунок 6.1. Попытка изменить элемент кортежа

Задание 7. На рисунке 7: для списка из 10 чисел определены количество элементов (10), сумма (177), минимум (4), максимум (42) и среднее арифметическое (17.7). Функция sorted() вернула новый отсортированный список, при этом исходный список остался без изменений. Метод sort()

изменил сам список и вернул None. Различие состоит в том, что sorted() создаёт новый результат, а list.sort() изменяет существующий список.

```
numbers = [4, 8, 15, 16, 23, 42, 7, 19, 31, 12]
print("Количество:", len(numbers))
print("Сумма:", sum(numbers))
print("Минимум:", min(numbers))
print("Максимум:", max(numbers))
print("Среднее:", sum(numbers) / len(numbers))

sorted_numbers = sorted(numbers)
print("sorted():", sorted_numbers)
print("Исходный список:", numbers)

numbers.sort()
print("После sort():", numbers)
```

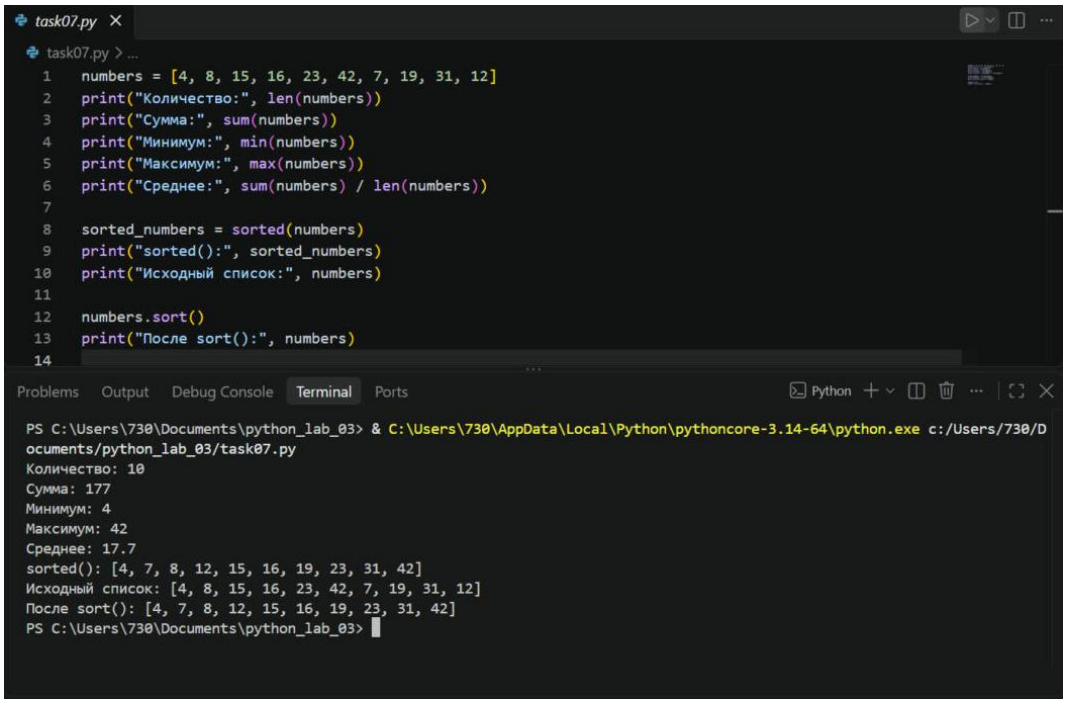


Рисунок 7. Встроенные функции для работы со списком

Задание 8. На рисунке 8: функции calculate_sum(), find_min() и find_max() реализованы без использования встроенных функций sum(), min() и max(): сумма накапливается в цикле, а минимум и максимум определяются

последовательным сравнением элементов. Результаты собственных функций (177, 4 и 42) совпадают с результатами встроенных функций Python.

```
def calculate_sum(numbers):
    total = 0
    for number in numbers:
        total += number
    return total


def find_min(numbers):
    result = numbers[0]
    for number in numbers:
        if number < result:
            result = number
    return result


def find_max(numbers):
    result = numbers[0]
    for number in numbers:
        if number > result:
            result = number
    return result


numbers = [4, 8, 15, 16, 23, 42, 7, 19, 31, 12]
print("calculate_sum:", calculate_sum(numbers), "| sum:", sum(numbers))
print("find_min:", find_min(numbers), "| min:", min(numbers))
print("find_max:", find_max(numbers), "| max:", max(numbers))
```

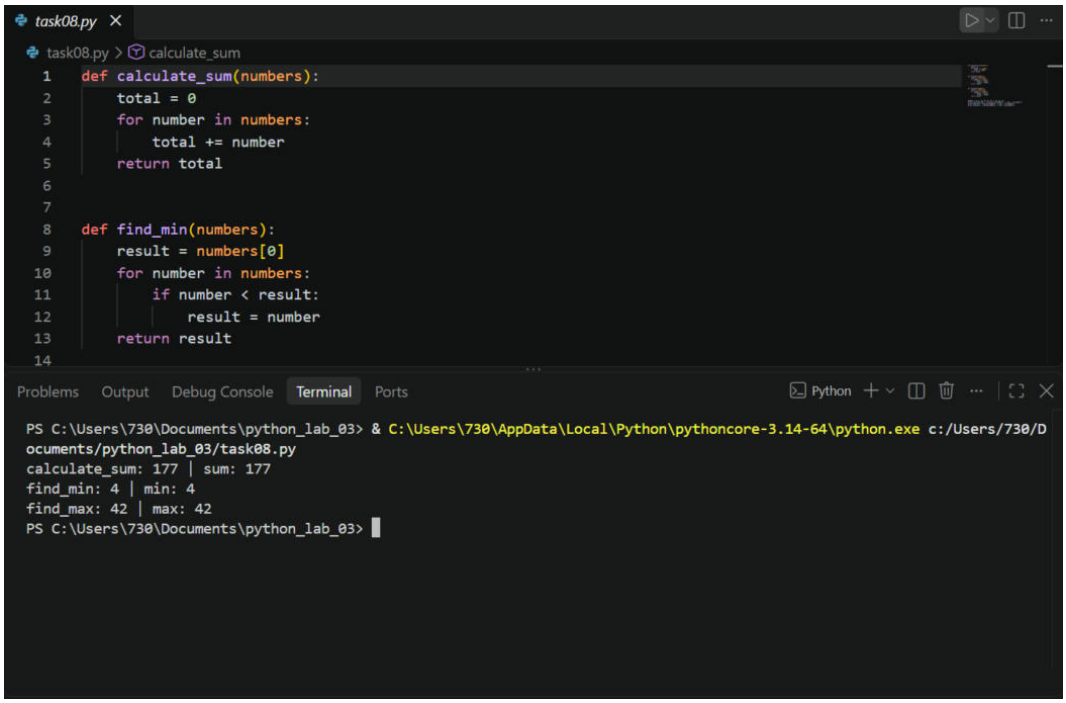


Рисунок 8. Собственные алгоритмы суммы, минимума и максимума

Задание 9. На рисунке 9: функция find_element() последовательно просматривает список и возвращает индекс первого найденного элемента. Для существующего значения 23 возвращён индекс 4, для отсутствующего значения 99 возвращено -1.

```
def find_element(numbers, target):
    for i in range(len(numbers)):
        if numbers[i] == target:
            return i
    return -1


numbers = [4, 8, 15, 16, 23, 42, 7, 19, 31, 12]
print(find_element(numbers, 23))
print(find_element(numbers, 99))
```

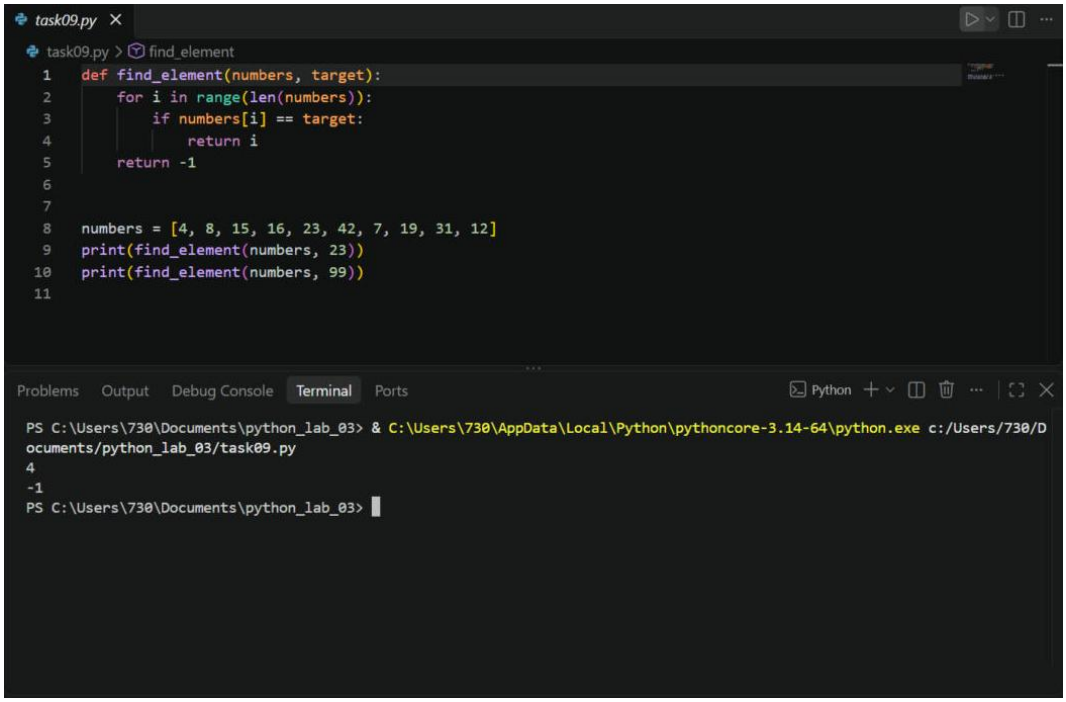


Рисунок 9. Линейный поиск

Задание 10. На рисунке 10: функция analyze() возвращает кортеж из трёх значений — минимума, максимума и среднего арифметического: (4, 42, 17.7).

Возвращённый кортеж распакован в три переменные minimum, maximum и average, значения которых выведены на экран.

```
def analyze(numbers):
    total = 0
    minimum = numbers[0]
    maximum = numbers[0]
    for number in numbers:
        total += number
        if number < minimum:
            minimum = number
        if number > maximum:
            maximum = number
    return minimum, maximum, total / len(numbers)


numbers = [4, 8, 15, 16, 23, 42, 7, 19, 31, 12]
result = analyze(numbers)
print("Кортеж:", result)

minimum, maximum, average = result
print("Минимум:", minimum)
print("Максимум:", maximum)
print("Среднее:", average)
```

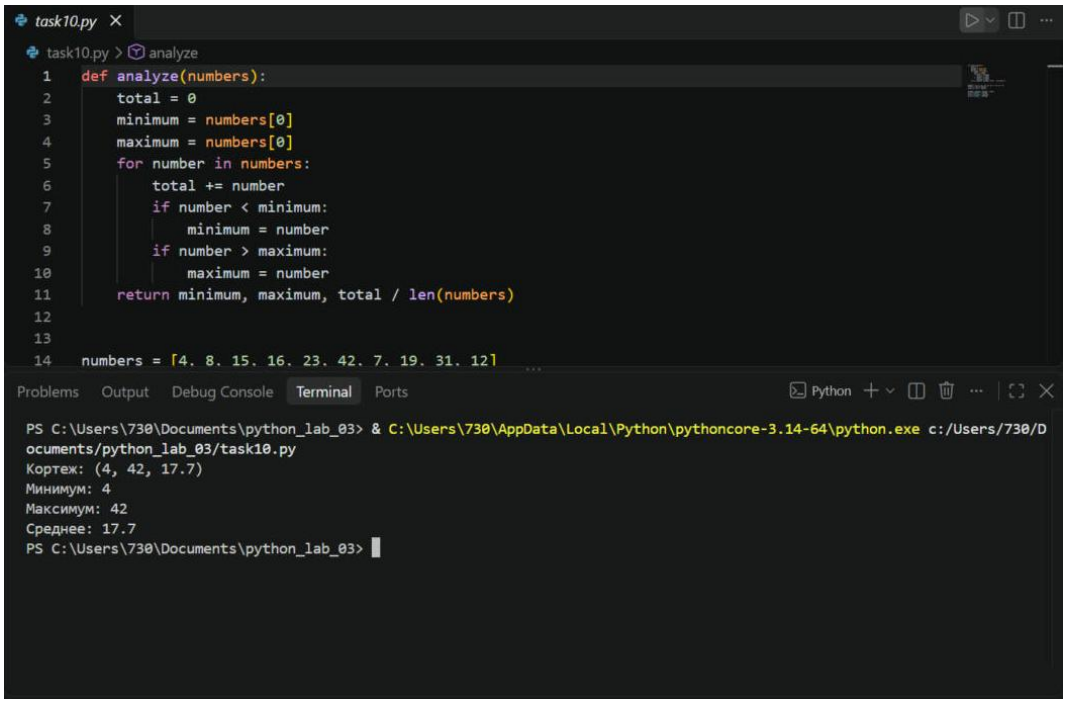


Рисунок 10. Возврат нескольких значений из функции

Задание 11. На рисунке 11: обычная сортировка списка [-10, 3, -2, 8, -5] функцией sorted() даёт результат [-10, -5, -2, 3, 8]. Сортировка с параметром key=lambda x: abs(x) выполняется по модулю чисел и даёт результат [-2, 3, -5, 8, -10]. Lambda-функция здесь задаёт правило, по которому sorted()

сравнивает элементы, и позволяет описать это правило в одной строке без отдельного определения через def.

```
numbers = [-10, 3, -2, 8, -5]
print(sorted(numbers))
print(sorted(numbers, key=lambda x: abs(x)))
```

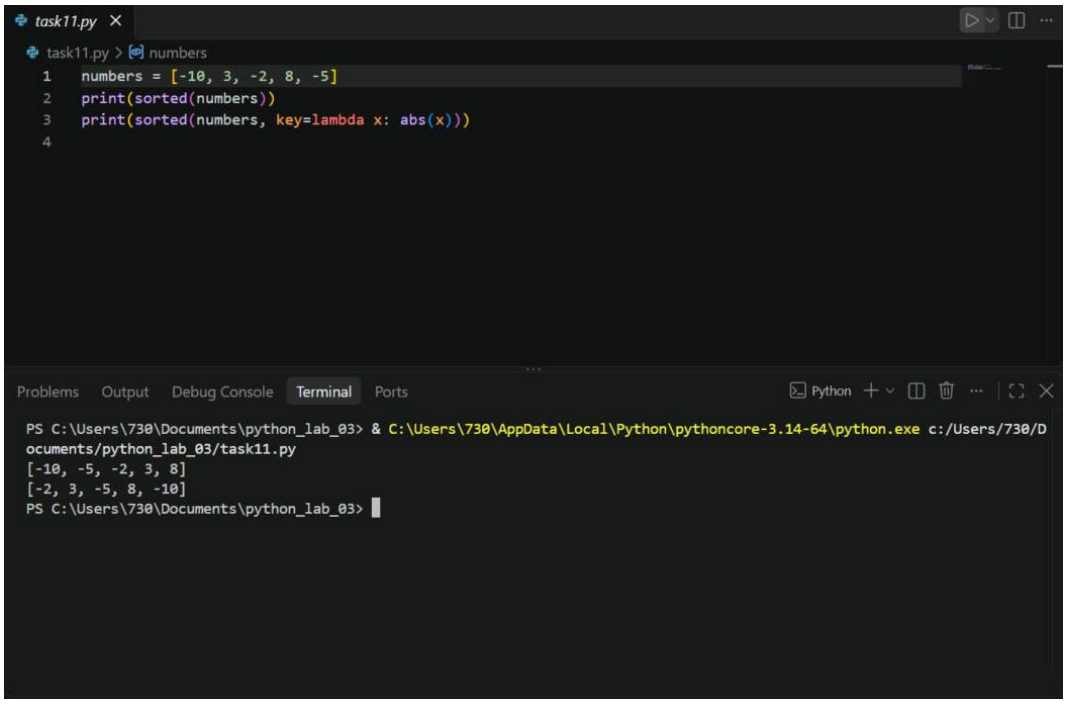


Рисунок 11. Сортировка с использованием lambda-функции

Задание 12. На рисунках 12 и 12.1: разработана программа анализа результатов измерений. Данные хранятся в списке из 10 значений. Реализованы функции calculate_average(), find_minimum(), find_maximum(), count_above_average(), find_value() и analyze_measurements(); в первых трёх функциях не используются встроенные sum(), min() и max(). Функция analyze_measurements() возвращает кортеж из минимума, максимума и среднего значения, который распаковывается в три переменные. Программа вывела: количество измерений — 10, минимум — 11.8, максимум — 16.4,

среднее значение — 13.97, количество значений выше среднего — 5, индекс искомого значения 14.7 — 5.

```
def calculate_average(measurements):
    total = 0
    for value in measurements:
        total += value
    return total / len(measurements)


def find_minimum(measurements):
    result = measurements[0]
    for value in measurements:
        if value < result:
            result = value
    return result


def find_maximum(measurements):
    result = measurements[0]
    for value in measurements:
        if value > result:
            result = value
    return result


def count_above_average(measurements):
    average = calculate_average(measurements)
    count = 0
    for value in measurements:
        if value > average:
            count += 1
    return count


def find_value(measurements, target):
    for i in range(len(measurements)):
        if measurements[i] == target:
            return i
    return -1


def analyze_measurements(measurements):
    return find_minimum(measurements), find_maximum(measurements),
calculate_average(measurements)


measurements = [12.5, 14.1, 13.8, 15.2, 12.9, 14.7, 13.3, 15.0, 16.4, 11.8]
target = 14.7

minimum, maximum, average = analyze_measurements(measurements)
print("Количество измерений:", len(measurements))
print("Минимум:", minimum)
print("Максимум:", maximum)
print(f"Среднее значение: {average:.2f}")
print("Выше среднего:", count_above_average(measurements))
```

print(f"Индекс значения {target}:", find_value(measurements, target))

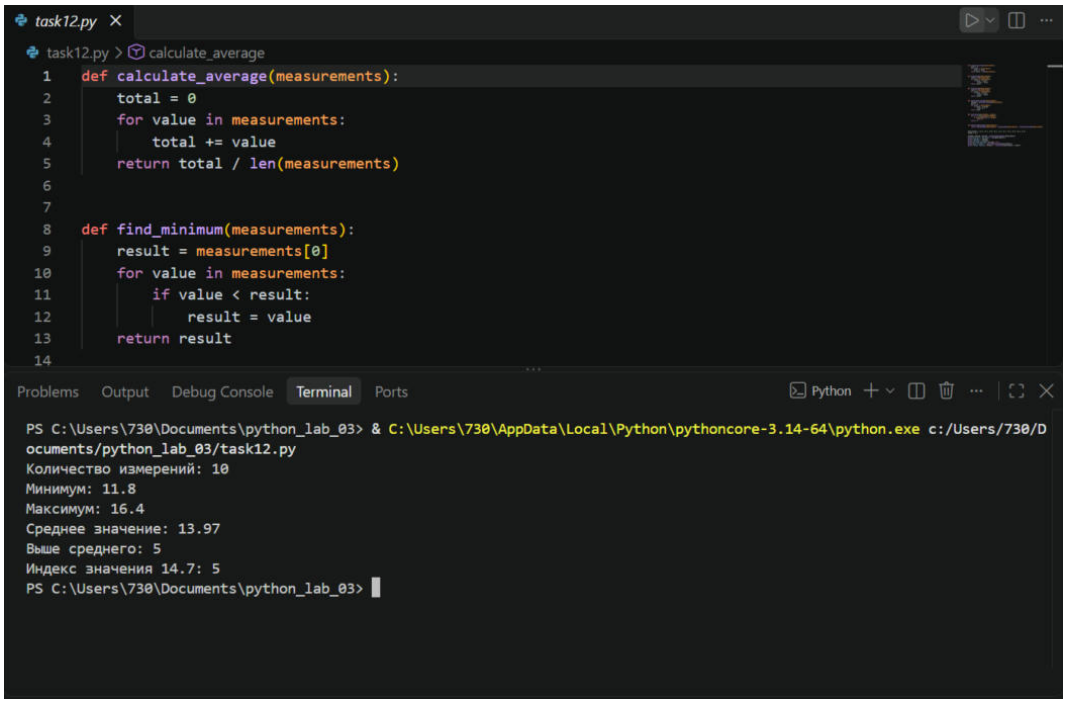


Рисунок 12. Анализ измерений, функции программы

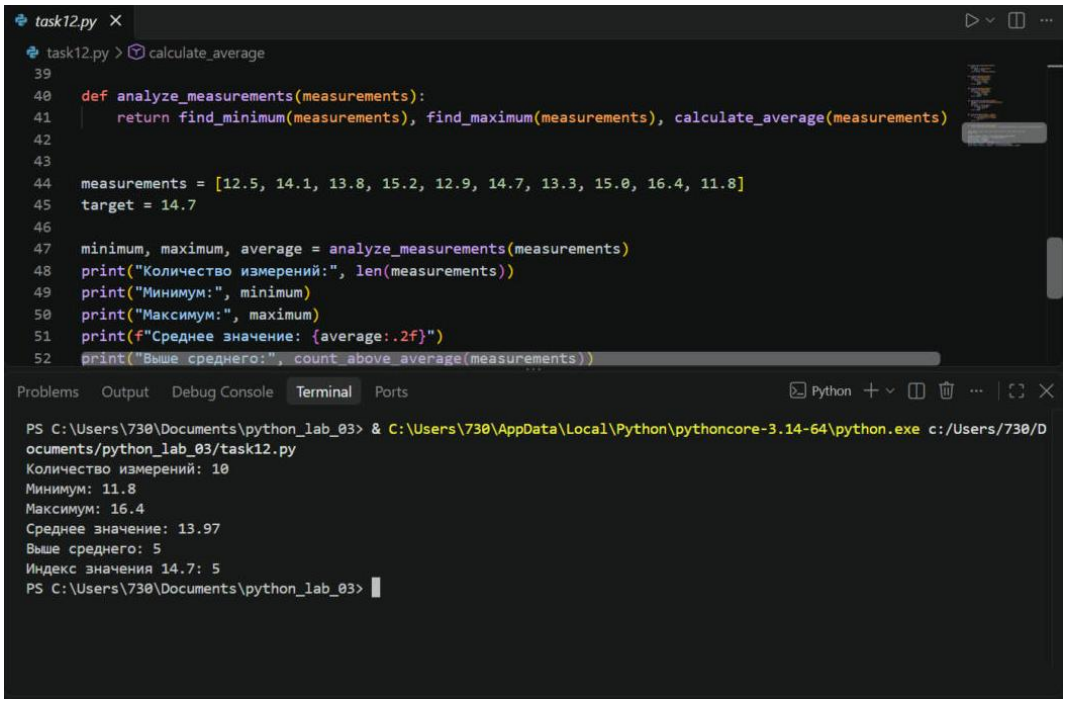


Рисунок 12.1. Анализ измерений, основная часть и результат

Вывод

В ходе лабораторной работы были изучены принципы создания и использования функций в языке Python: определение функции с помощью def, вызов функции, параметры и аргументы, оператор return. Показано различие между print() и return: print() выводит значение на экран, а return передаёт

результат в место вызова. Использованы аргументы по умолчанию на примере параметра unit функции describe_rectangle().

Рассмотрены области видимости переменных: локальная переменная существует только внутри функции, не изменяет глобальную переменную с тем же именем и недоступна после завершения функции, что подтвердила ошибка NameError.

Освоена работа со списками и кортежами: индексация, отрицательные индексы, срезы, методы append() и remove(), распаковка и преобразование типов. Выяснено, что список является изменяемым, а кортеж — неизменяемым, и попытка изменить его элемент приводит к ошибке TypeError. Использованы встроенные функции len(), sum(), min(), max() и sorted(); показано различие между sorted() и методом sort().

Реализованы собственные алгоритмы суммирования, поиска минимума и максимума, линейного поиска и подсчёта значений выше среднего, а также применена lambda-функция в качестве параметра key функции sorted(). В самостоятельной работе разработана программа анализа набора результатов измерений, состоящая из отдельных функций.